

Hydra Config Framework

Complete Tutorial — 9 Lessons

Lesson 0 Basics & decorator

Lesson 1 Config groups

Lesson 2 Override syntax

Lesson 3 Interpolation & resolvers

Lesson 4 Structured configs

Lesson 5 `instantiate()`

Lesson 6 Multirun sweeps

Lesson 7 Advanced patterns

Lesson 8 Real-world ML project

Generated from your `hydra_learning_setup_final.py` — all 17 tests passing

Basics

Run the app (default config)

```
cd 00_basics
python app.py
```

Override any value from the CLI

```
python app.py db.host=prod-server db.port=5433
python app.py training.lr=0.01 training.epochs=50
python app.py training.device=cuda
```

Print config without running the app

```
python app.py --cfg job
```

Key facts

- `cfg` is read-only by default (OmegaConf struct mode).
- Dot notation: `cfg.db.host` — same as `cfg["db"]["host"]`
- Types are preserved from YAML: `port=5432` is int, `lr=0.001` is float.
- Hydra auto-creates `outputs/YYYY-MM-DD/HH-MM-SS/` and `chdirs` there.

Your script's CWD is that outputs dir when running.

Read next

For a slower explanation of exactly which code/config lines create the behavior in this lesson, read:

```
cat CODE_WALKTHROUGH.md
```

Config Groups

Default (postgres + small + adam)

```
cd 01_config_groups
python train.py
```

Swap the database

```
python train.py db=sqlite
python train.py db=mysql
```

Swap model size

```
python train.py model=large
python train.py model=medium
```

Mix and match freely

```
python train.py db=sqlite model=large optimizer=adamw
python train.py db=mysql model=medium optimizer=sgd experiment_name=my_exp
```

Override a value **within** the chosen group

```
python train.py model=large model.dropout=0.05
python train.py optimizer=adam optimizer.lr=3e-4
```

Print which config files were loaded

```
python train.py --info defaults
```

Key concepts

- Config group identifier = folder name, e.g. `db` , `model` , `optimizer` .
- Config option identifier = file name without `.yaml` , e.g. `postgres` .
- Identifiers usually are not inside the YAML file; Hydra discovers them from paths.
- `# @package _global_` means merge the file at the root of the final config tree.

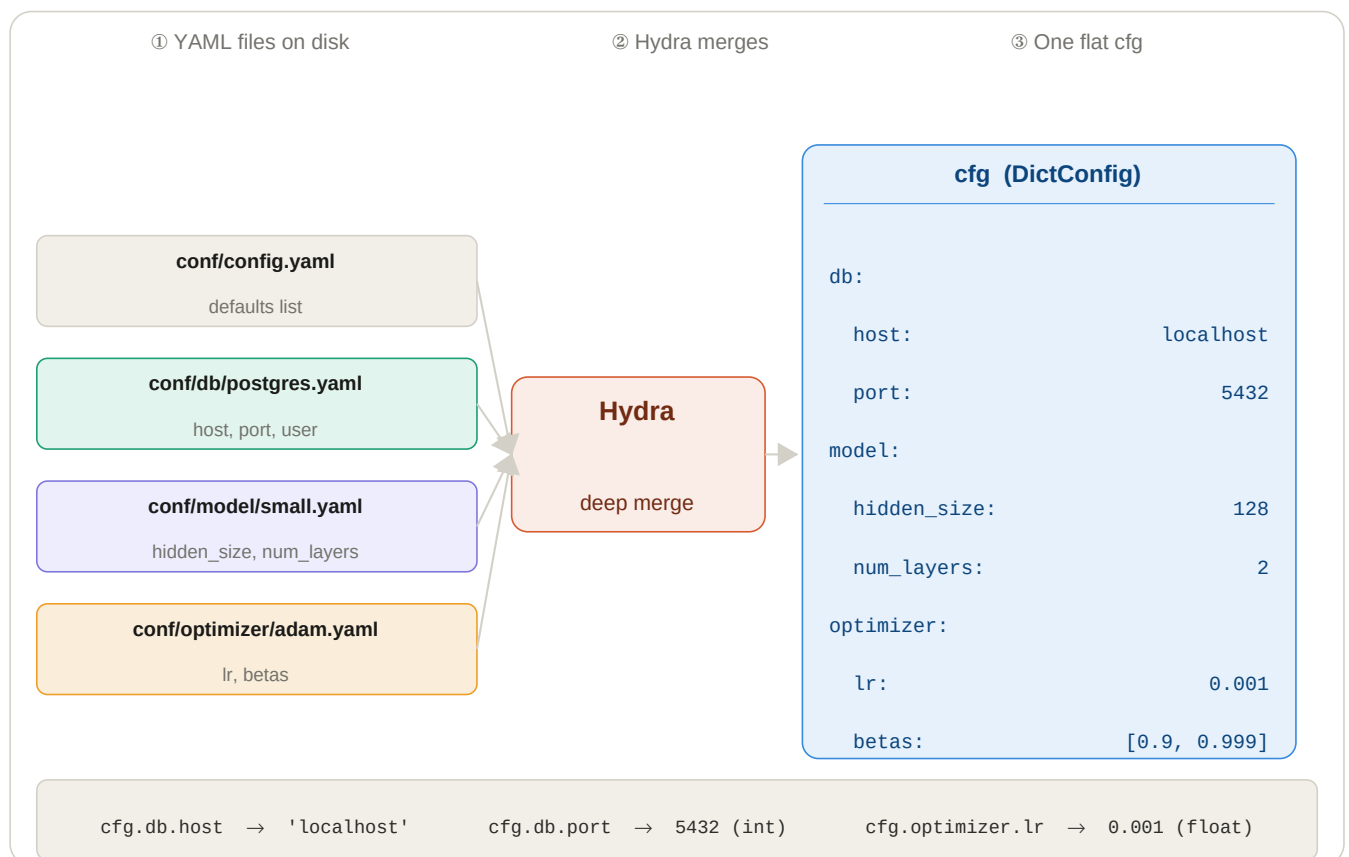
- Without `__global__`, Hydra normally places a group config under its group path.
- `__self__` controls merge precedence, not hierarchy. Later values win on overlap.
- Avoid overlapping ownership: let `optimizer/` own `optimizer.*`, `model/` own `model.*`, etc.

Read next

For a slower explanation of exactly which code/config lines create the behavior in this lesson, read:

```
cat CODE_WALKTHROUGH.md
```

How Hydra assembles the final config



Override Syntax

Run all commands from `02_overrides/` .

Shell quoting equivalences

These pairs produce the same Hydra config in bash/zsh:

```
python demo.py 'app.tags=[staging,v2,hotfix]'
python demo.py app.tags="[staging,v2,hotfix]"
```

```
python demo.py 'app.name="hello:world"'
python demo.py app.name="hello:world"
python demo.py 'app.name=hello:world'
```

```
python demo.py '+server={host:127.0.0.1,port:9090,ssl:true}'
python demo.py +server="{host:127.0.0.1,port:9090,ssl:true}"
```

The shell removes its own quote characters before Python receives the arguments. The quotes mostly protect brackets/braces/commas/colons from the shell.

`+` vs `++`

Operator	If key exists	If key does not exist	Meaning
<code>`key=value`</code>	Updates it	Error	Modify existing key only
<code>`+key=value`</code>	Error	Adds it	Insert new key only
<code>`++key=value`</code>	Updates it	Adds it	Upsert: update or insert

This table is easiest to understand for leaf paths like `app.name` .

Whole-dictionary overrides such as `+server={...}` can merge/update dictionary contents while adding new subkeys.

Try:

```
python demo.py app.name=new_name      # works: existing key
python demo.py +app.version=2.0       # works: new key
python demo.py ++app.version=2.0      # works: new key
python demo.py ++app.name=new_name    # works: existing key
```

These should fail:

```
python demo.py +app.name=new_name      # app.name already exists
python demo.py app.version=2.0          # app.version does not exist
```

Inline dictionary syntax

Inside Hydra dictionaries, use `:` between key and value:

```
python demo.py '+server={host:127.0.0.1,port:9090,ssl:true}'
python demo.py +server="{host:127.0.0.1,port:9090,ssl:true}"
```

This works as a dictionary merge/update on `server` : existing fields

like `host` and `port` change, and the new `ssl` field is added.

This is wrong inside the dictionary:

```
python demo.py '+server={host:127.0.0.1,port:9090,ssl=true}'
```

For leaf keys, `+` is stricter:

```
python demo.py +app.name=new_name      # fails because app.name exists
python demo.py +server.host=127.0.0.1  # fails because server.host exists
```

More examples

```
python demo.py '~limits.max_retries'
python demo.py app.debug=null
python demo.py 'app.tags=[staging,v2,hotfix]'
python demo.py --cfg job
python demo.py --cfg hydra
python demo.py --cfg all
```

`--cfg job` means: print the application/job config and exit.

Read next

For a slower explanation of exactly which code/config lines create the behavior in this lesson, read:

```
cat CODE_WALKTHROUGH.md
```

Interpolation & Resolvers

Basic run

```
cd 03_interpolation
python app.py
```

Lazy means formula-like

```
base_dir: /workspace
output_dir: ${base_dir}/outputs/${project}
```

`output_dir` is not permanently computed when the config is loaded.

It is resolved when you access `cfg.output_dir`.

```
python app.py base_dir=/data/experiments
```

`data_dir`, `output_dir`, `checkpoint_dir`, and `model.save_path` update because they reference `base_dir`.

`oc.env` means OS environment variable

```
api_key: ${oc.env:API_KEY,not_set}
```

`oc.env` is a built-in OmegaConf resolver. It reads the OS environment.

`${project}` reads from the config tree.

```
API_KEY=mysecretkey python app.py
```

Custom resolver: `choose\_log\_level`

```
env: development
log_level: ${choose_log_level:${env}}
```

This calls the Python function registered in `app.py`:

```
lambda env: "DEBUG" if env == "development" else "INFO"
```

It does not select another config file.

```
python app.py env=production
python app.py env=development
```

Deleting an interpolation target

```
python app.py '~project'
```

`~project` deletes the key. Then `run_name` tries to resolve `${project}` and errors when accessed. `-m` is not needed here.

`oc.select` for safe access

```
safe_value: ${oc.select:maybe.missing.key,fallback_value}
```

This returns `fallback_value` instead of crashing if the key is missing.

Read next

For a slower explanation of exactly which code/config lines create the behavior in this lesson, read:

```
cat CODE_WALKTHROUGH.md
```


Structured Configs

Structured configs are optional. They can be overkill for fast PhD research code, but they are useful when config mistakes waste time.

Main idea

```
dataclass schema      = allowed structure + types + default values
ConfigStore.instance() = register schema with Hydra
defaults: - app_schema = actually use the schema
YAML values      = override schema defaults
```

What `AppConfig` does

`AppConfig` describes the whole final config tree:

```
@dataclass
class AppConfig:
    db: DBConfig
    model: ModelConfig
    optimizer: OptimizerConfig
    training: TrainingConfig
    experiment_name: str
    seed: int
```

It tells Hydra that `training.epochs` must be an int, `optimizer.lr` must be a float, ordinary unknown overrides should fail.

Why add it to `config.yaml`?

This registers the schema:

```
cs.store(name="app_schema", node=AppConfig)
```

But registration alone does not activate it.

This activates it:

```
defaults:
- app_schema
- _self_
```

It means: start with `AppConfig` defaults, then merge this YAML file on top.

Without `- app_schema`, Hydra treats the file as plain YAML, so `training.epochs=not_an_int` can become a string.

Run

```
cd 04_structured_configs
python app.py
```

Type mismatch

```
python app.py training.epochs=50
python app.py training.epochs=not_an_int
```

The second command should fail when the schema is active.

MISSING fields

In the schema:

```
experiment_name: str = MISSING
```

means the value must be provided by YAML or CLI.

This intentionally fails:

```
python app.py --config-name config_missing
```

Unknown keys

Built-in structured config validation catches ordinary unknown-key overrides:

```
python app.py training.nonexistent=1
```

Important nuance: Hydra's `+` and `++` operators are designed to append or upsert keys, so they can intentionally add keys even when a schema is active.

This tutorial adds an optional Python-side strict validator:

```
strict_no_extra_keys: true
```

Therefore this also fails in this lesson:

```
python app.py +training.nonexistent=1
```

To see Hydra's default append behavior, disable the extra guard:

```
python app.py strict_no_extra_keys=false +training.nonexistent=1
```

Check schema activation with:

```
python app.py --info defaults
```

You should see `app_schema` loaded before `config` .

Plain YAML comparison

This intentionally does not activate the schema:

```
python app.py --config-name config_plain training.epochs=not_an_int
```

In plain YAML mode, Hydra does not know that epochs should be int.

Recommendation

For most research code, start with YAML-only Hydra plus manual sanity checks. Add structured configs later for stable sections like training, optimizer, checkpointing, and logging.

Read next

For a slower explanation of exactly which code/config lines create the behavior in this lesson, read:

```
cat CODE_WALKTHROUGH.md
```

instantiate()

Default run (Adam optimizer + Cosine scheduler)

```
cd 05_instantiate
python app.py
```

Swap optimizer — the class that gets built changes

```
python app.py optimizer=sgd
```

Swap scheduler

```
python app.py scheduler=step
```

Override a constructor arg

```
python app.py optimizer.lr=1e-5
python app.py model.hidden_size=512 model.dropout=0.3
```

Key: `_target_` format

`_target_` must be a fully-qualified Python dotted path:

```
_target_: mypackage.submodule.MyClass
_target_: torch.optim.Adam           # from torch
_target_: torch.nn.CrossEntropyLoss  # etc.
```

`_recursive_` (default True)

If your config has nested objects that also have `_target_`,

Hydra instantiates them all automatically (depth-first).

`_partial_=True`

Returns a `functools.partial` — call it later with remaining args.

Perfect for optimizers: define `lr/weight_decay` in config,

pass `model.parameters()` in code.

```
opt_fn = instantiate(cfg.optimizer, _partial_=True)
optimizer = opt_fn(params=model.parameters())
```

`_convert_="all"`

Converts OmegaConf containers to plain Python dicts/lists

before passing to the constructor. Use when your class expects

plain Python types, not OmegaConf wrappers.

Registry vs Hydra instantiate

You do not need to use `_target_` everywhere. For research ML templates,

a Registry is often easier to debug for models, datasets, losses, and

metrics:

```
model:
  name: resnet50
optimizer:
  name: adamw
  lr: 0.0003
```

```
model = MODEL_REGISTRY.build(cfg.model.name, cfg.model)
optimizer = OPTIMIZER_REGISTRY.build(
    cfg.optimizer.name,
    model.parameters(),
    cfg.optimizer,
)
```

Good practical split:

```
Registry:      models, datasets, losses, metrics, trainers
instantiate(): optional; useful for stable third-party constructors
```

Read next

For a slower explanation of exactly which code/config lines create

the behavior in this lesson, read:

```
cat CODE_WALKTHROUGH.md
```

Multirun Sweeps

Run all commands from `06_multirun/` .

Normal runs do not need `-m``

```
python train.py
python train.py model=large
python train.py optimizer.lr=0.0003
python train.py model=large optimizer=sgd
```

Use `-m`` for sweeps

```
python train.py -m optimizer.lr=0.1,0.01,0.001
```

This launches 3 jobs.

Cartesian product

```
python train.py -m model=small,large optimizer=adam,sgd
```

This launches $2 \times 2 = 4$ jobs.

Groups + values

```
python train.py -m model=small,large optimizer.lr=0.1,0.01,0.001
```

This launches $2 \times 3 = 6$ jobs.

Range syntax

```
python train.py -m 'optimizer.lr=range(0.001,0.1,0.01)'
```

Values like `0.0110000000000000001` are normal floating-point behavior.

Glob syntax

```
python train.py -m 'model=glob(*)'
```

This sweeps every config file in `conf/model/` . If the folder only has `small.yaml` and `large.yaml` , it launches 2 jobs.

Hydra logging

These lines are Hydra's default logging, not code you wrote:

```
[HYDRA] Launching 4 jobs locally
[HYDRA]      #0 : model=small optimizer=adam
```

Your script's output starts at `[Run ...]` .

Hydra logging comes from internal configs such as:

```
hydra/hydra_logging/default
hydra/job_logging/default
```

You can see them with:

```
python train.py --info defaults
```

Parallel multirun

```
python train.py -m hydra/launcher=joblib \
  hydra.launcher.n_jobs=4 \
  model=small,large optimizer=adam,sgd
```

Access current run info

```
from hydra.core.hydra_config import HydraConfig
hcfg = HydraConfig.get()
hcfg.job.num
hcfg.overrides.task
hcfg.runtime.output_dir
```

Read next

For a slower explanation of exactly which code/config lines create the behavior in this lesson, read:

```
cat CODE_WALKTHROUGH.md
```


Advanced Patterns

Run the main app

```
cd 07_plugins
python app.py
```

Add optional config blocks (append groups)

```
python app.py '+extra=[logging,callbacks]'
```

Use the compose API (no decorator — great for tests)

```
python advanced_patterns.py
```

@package directive quick reference

In any YAML file, the first line can be a package directive:

```
# @package _global_      ← merge at root of config tree
# @package model.encoder ← merge under model.encoder
# @package _group_       ← merge under the group name (default)
```

Without a directive, Hydra uses the group name as the package.

get_original_cwd() — critical for file paths!

Hydra CHANGES the working directory to the run output dir.

This breaks relative file paths. Use:

```
from hydra.utils import get_original_cwd, to_absolute_path
data_path = to_absolute_path(cfg.data_root) # always correct
```

Overriding Hydra's own config

Hydra has an internal config tree (hydra.*) you can override:

```
python app.py hydra.run.dir=/my/output
python app.py hydra.job.name=my_job
python app.py hydra.verbose=true      # verbose logging
python app.py 'hydra.output_subdir=null' # disable .hydra/ subdir
```

Disabling config output dir

```
python app.py hydra.output_subdir=null hydra.run.dir=.
```

This makes Hydra NOT change the CWD and NOT create output dirs.

Useful for interactive scripts.

Read next

For a slower explanation of exactly which code/config lines create the behavior in this lesson, read:

```
cat CODE_WALKTHROUGH.md
```

Real-World ML Project

All commands from 08_real_world_ml/.

Basic run (experiment_name is required — use ??? in YAML)

```
python train.py experiment_name=baseline
```

Swap dataset + model

```
python train.py experiment_name=cifar_run dataset=cifar10 model=resnet
```

Swap logger (e.g. to WandB)

```
python train.py experiment_name=wandb_test logger=wandb
```

Full sweep: dataset × model × optimizer (8 runs)

```
python train.py -m \
  experiment_name=big_sweep \
  dataset=mnist,cifar10 \
  model=cnn,resnet \
  optimizer=adamw,sgd
```

Cross-config interpolation

Notice in conf/model/cnn.yaml:

```
num_classes: ${dataset.num_classes}
```

Hydra resolves this from the currently-loaded dataset config!

Switch dataset and num_classes updates automatically.

Override scheduler T_max (which interpolates from training.epochs)

```
python train.py experiment_name=long training.epochs=200
```

The scheduler's T_max automatically updates to 200.

The ??? pattern (required fields)

experiment_name: ??? means Hydra errors if not provided.

This prevents accidentally running with a missing name.

```
python train.py # ← ERROR: experiment_name is missing
python train.py experiment_name=my_run # ← OK
```

Read next

For a slower explanation of exactly which code/config lines create the behavior in this lesson, read:

```
cat CODE_WALKTHROUGH.md
```